

第15课 LLM Agents

北京交通大学软件学院

严禁复制



目录

1. Agent的定义：从经典AI到LLM Agent
2. Agent的分类：多维度剖析LLM Agent
3. 典型Agent详解：
 1. 功能、应用场景
 2. 技术难点与挑战
 3. 现有解决方案与架构
4. Agent编程实践

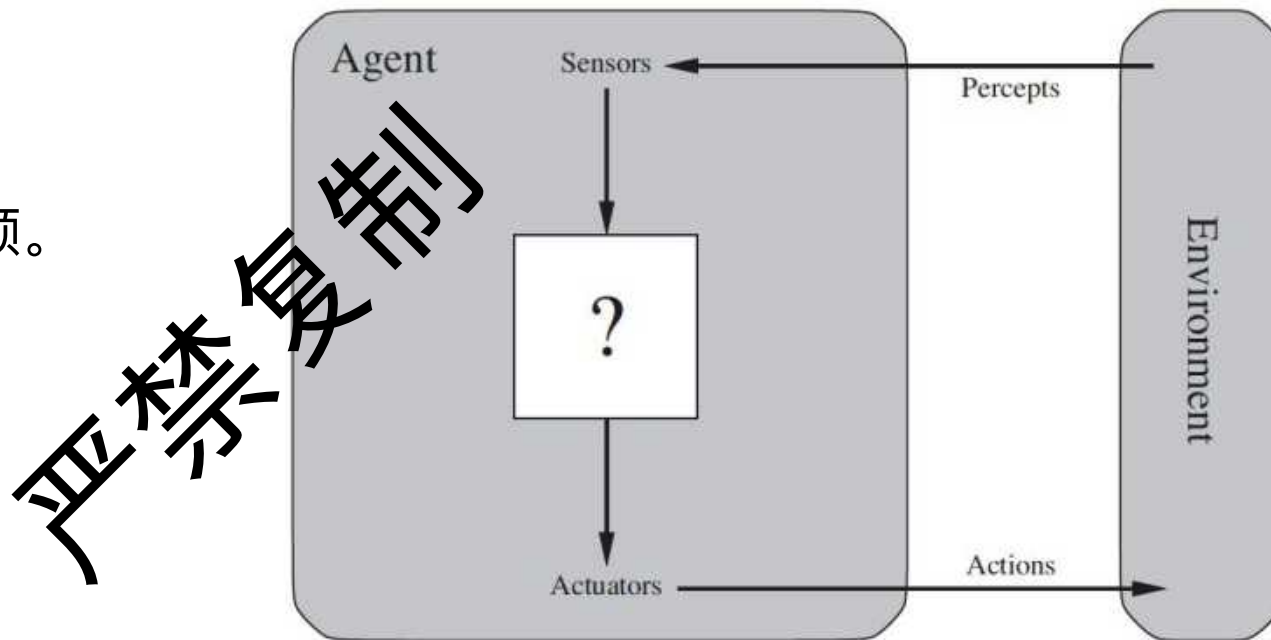
严禁复制



01 复制 Agent 的定义

什么是Agent? (经典AI视角)

- Agent (智能体) 指能够感知环境并通过执行器作用于环境的任何事物。
(Russell & Norvig, 2020)
- 核心特征:
 - 自主性: 独立操作, 无需人类干预。
 - 反应性: 对环境变化做出响应。
 - 主动性: 主动发起目标导向行为。
 - 社会性: 与其他Agent交互。
- 理性Agent:
 - 给定一个收益度量的方法
 - Agent对于特定的感知序列, 根据先验知识, 选择一个行为, 产生最大化收益。
 - 例如: 出租车司机



LLM Agent是LLM使用方式的扩展



LLM Agent 的核心特征

- 推理与规划能力：
 - 利用LLM进行复杂逻辑推演。
- 工具使用能力：
 - 调用外部API、数据库等工具扩展能力。
- 记忆能力：
 - 短期记忆（上下文）
 - 长期记忆（知识库）。
- 自我反思与改进：
 - 评估自身行为并进行优化。

严禁复制



02 Agent的分类

复制

LLM Agent 分类维度

- LLM Agent的分类标准多样，尚无统一体系。
- 常见分类维度包括：
 - 使用哪种类型的LLM
 - 应用领域和任务类型
 - Agent的架构设计
- 参考：
 - Wang et al. (2023). A survey on Large Language Model based Autonomous Agents.

禁止复制

基于核心LLM能力的分类

- 单任务 vs. 通用任务Agent:
 - 前者针对特定问题优化，后者追求广泛适用性。
- 单模态 vs. 多模态Agent:
 - 前者仅处理文本，后者融合图像、语音等信息。
- 封闭 vs. 开放模型Agent:
 - 前者基于特定API（如GPT系列），后者基于开源模型。

禁止转载

基于应用领域和任务类型

- 对话型Agent：
 - 聊天机器人、虚拟助手 (Siri, Alexa)。
- 任务导向型Agent：
 - 自动化办公、软件开发、科学研究。
- 具身Agent (Embodied Agents)：
 - 控制机器人与物理世界交互。
- 游戏AI Agent：
 - 在虚拟环境中扮演角色，与玩家互动。
- 教育Agent：
 - 个性化辅导，智能答疑。

禁止复制

基于Agent架构设计的分类

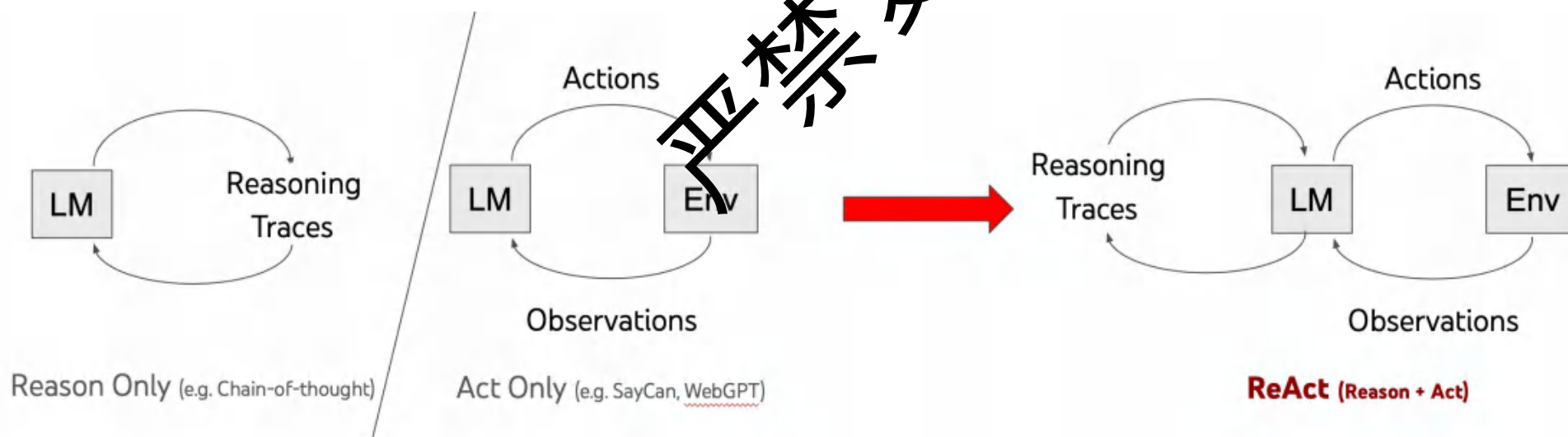
分类	特点
简单反应式Agent (LLM as Core Controller):	直接将用户输入prompt给LLM，LLM输出即为行动。
规划型Agent (Planning Agents):	LLM分解任务，制定多步计划，并逐步执行。 例子：ReAct, Plan-and-Solve。
工具使用型Agent (Tool-augmented Agents):	LLM学习调用外部工具来获取额外信息或执行动作。 例子：Toolformer, Gorilla。
反思型Agent (Reflective Agents):	LLM对自身输出或执行结果进行评估和修正。 例子：Self-Refine, Reflexion。
记忆增强型Agent (Memory-augmented Agents):	集成短期和长期记忆机制，支持持续学习和个性化。
多Agent系统 (Multi-Agent Systems, MAS):	多个LLM Agent协同工作，完成复杂任务。 例子：CAMEL, MetaGPT。



03 复制 亚林哥类Agent的详细定义

类型一：规划型Agent – ReAct框架

- 核心思想 (ReAct: Reason + Act):
 - 思考: LLM生成一个步骤, 下一步做什么
 - 行动: 根据思考采取一个行动
 - 观察: 观察返回的结果
 - 再次思考: 生成一个新的步骤。



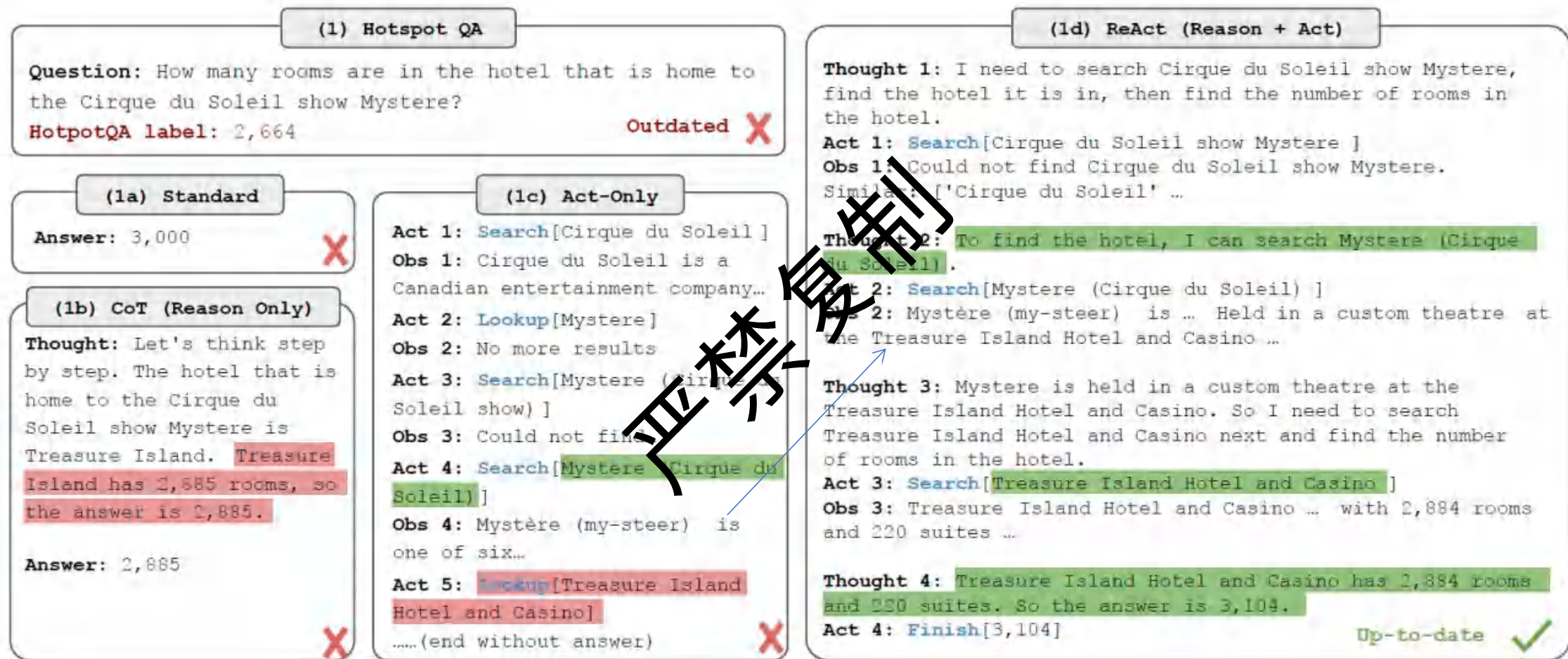
COT

- 核心思想：CoT 的核心思想是引导 LLM 在给出最终答案之前，先生成一系列中间的、连贯的推理步骤。这就像我们解决复杂问题时，会先在草稿纸上一步步演算和思考一样。
- “Roger 有 5 个网球。他又买了 2 罐网球，每罐有 3 个。他总共有多少个网球？”
- 没有 CoT 的回答：
 - 11 个。
- 有 CoT 的回答可能是：
 - “Roger 开始有 5 个球。”
 - “他买了 2 罐，每罐 3 个，所以他新买了 $2 * 3 = 6$ 个球。”
 - “所以他总共有 $5 + 6 = 11$ 个球。”
 - “答案是 11。”

ReAct (Reason and Act, 推理与行动)

- 核心思想：
 - ReAct 的核心思想是让 LLM 不仅能够进行内部思考 (Reason) ,
 - 还能够与外部环境或工具进行交互以获取额外信息或执行操作 (Act) ,
 - 然后根据交互结果再次思考,
 - 形成一个“思考-行动-观察”的循环。
- 例子：“目前法国的首都是哪里？它的人口是多少？”
 - Thought: 我需要先找到法国的首都。
 - Act: Search("法国的首都")
 - Observation: (搜索引擎返回) “法国的首都是巴黎。”
 - Thought: 我已经知道首都是巴黎了。现在我需要查找巴黎的人口。
 - Act: Search("巴黎的人口")
 - Observation: (搜索引擎返回) “巴黎市的人口约为 210 万 (截至 2023 年)。”
 - Thought: 我已经找到了所有需要的信息。
 - Answer: 法国的首都是巴黎，其人口约为 210 万。

ReAct



<https://zhuanlan.zhihu.com/p/624003116>

ReAct Agent – 功能与优势

- 功能：
 - 将复杂任务分解为可管理的小步骤。
 - 显式追踪推理过程，提高任务完成的可靠性。
 - 动态调整计划以应对意外情况。
- 优势：
 - 提高LLM在复杂任务上的表现，尤其需多步推理。
 - 增强了模型的可解释性，易于调试和理解。
 - 能与外部工具（如搜索引擎）结合，获取实时信息。

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

ReAct Agent – 适用场景

- 知识密集型问答和事实核查：
 - 复杂问答系统：
 - 处理需要多步检索、信息整合和逻辑推理才能得出答案的问题
 - 如HotpotQA，需要多步检索与推理。
 - 事实核查和信息验证：
 - 分解声明，检索证据，综合判断。
- 多工具的任务自动化和智能助手：
 - 智能助手：
 - 作为用户的智能代理，理解用户需求并利用外部工具
 - 理解用户需求，搜索商品，比较筛选。
 - 多工具的任务自动化：
 - 自动化执行涉及多步操作和外部系统交互的日常任务
 - 如预订会议室，需要查询日历并发送邀请。

ReAct Agent – 技术难点与解决方案

- 难点1：规划的幻觉 (Planning Hallucination)

- LLM可能生成无效或不相关的行动步骤。
- 解决方案：
 - 优化Prompt设计，提供明确的行动空间。
 - 引入验证机制，检查行动的合理性。
 - Fine-tuning模型，增强规划能力。

- 难点2：错误累积 (Error Propagation)

- 早期步骤的错误会影响后续决策。
- 解决方案：
 - 加入反思和修正环节。
 - 允许Agent从错误中学习并重新规划。

ReAct Prompt的例子

你是一个 ReAct 智能体，旨在通过交错的思考 (Thought)、行动 (Action) 和观察 (Observation) 步骤来完成任务。
你的目标是：{在此处填写具体任务目标，例如：“准确回答用户提出的问题”或“为用户预订一个符合要求的会议室”}

你的运作遵循以下循环：

1. ****思考 (Thought):**** 分析当前情况，制定整体计划，明确你需要哪些信息，以及下一步应采取什么行动来实现目标。
2. ****行动 (Action):**** 从可用工具中选择 ****一个**** 行动来收集信息或与环境互动。
3. ****观察 (Observation):**** 接收你行动的结果。

重复此循环，直到你拥有足够的信息来给出最终答案或完成任务。

****可用工具 (Available Tools):****

{在此处详细描述每个工具}

例如：

- ``Tool_Name_1[input_description]``：工具1的描述，它做什么，输入参数
- ``Tool_Name_2[input_description]``：工具2的描述。

****输出格式:****

轮到你时，你 ****必须**** 使用以下格式：

Thought: [你的推理过程和下一步计划。]

Action: [Tool_Name][工具的输入参数]

在你提供一个行动 (Action) 后，你将会收到一个观察结果 (Observation)。请使用这个观察结果来指导你的下一步思考。

****任务完成:****

当你认为你已经得到最终答案或完成了任务时，请使用以下特殊行动：

Action: Finish [最终答案或任务完成的确认信息。]

****重要指南:****

- 将问题分解为更小、可管理的步骤。
- 一步一步地思考。
- 有效利用工具收集必要信息。
- 如果一个行动没有产生预期的结果，思考原因并尝试不同的方法或修改行动。

****以下是一个示例:****

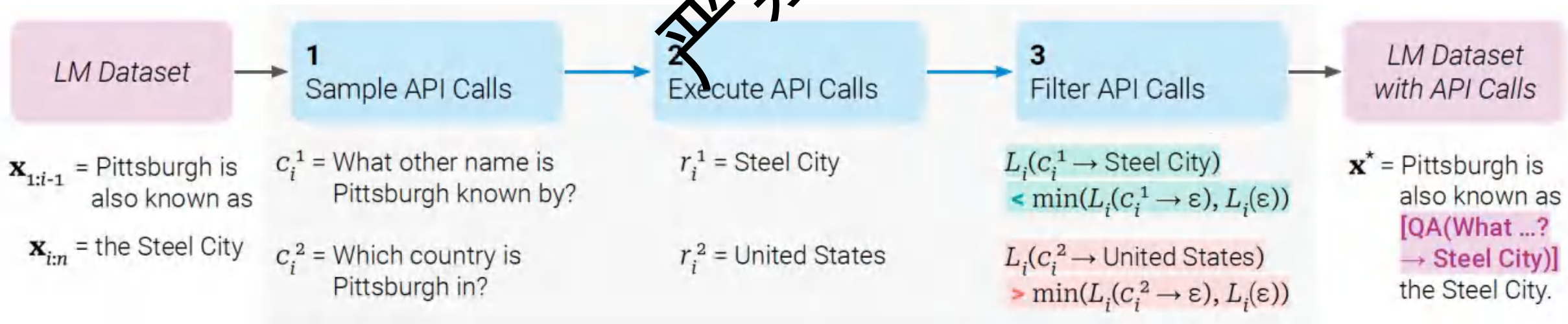
{在此处提供一个或多个完整的 Few-Shot 示例，展示从问题到最终答案的完整 Thought-Action-Observation 流程}

现在，让我们开始。

{初始问题或任务描述}

类型二：工具使用型Agent – Toolformer

- 定义：
 - 让大型语言模型（LLMs）通过一种自监督的方式学会自主调用外部工具。
- 核心思想：
 - 让LLM自主学习何时以及如何调用外部工具（API）。
 - 通过Self-Play方式生成工具使用训练数据。
- 论文： Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools.



Toolformer Agent – 功能与优势

- 功能：

- 无缝集成多种外部工具（计算器、搜索引擎、日历等）。
- 通过API调用扩展LLM自身不具备的能力。
- 动态决定是否需要使用工具以及使用哪个工具。

- 优势：

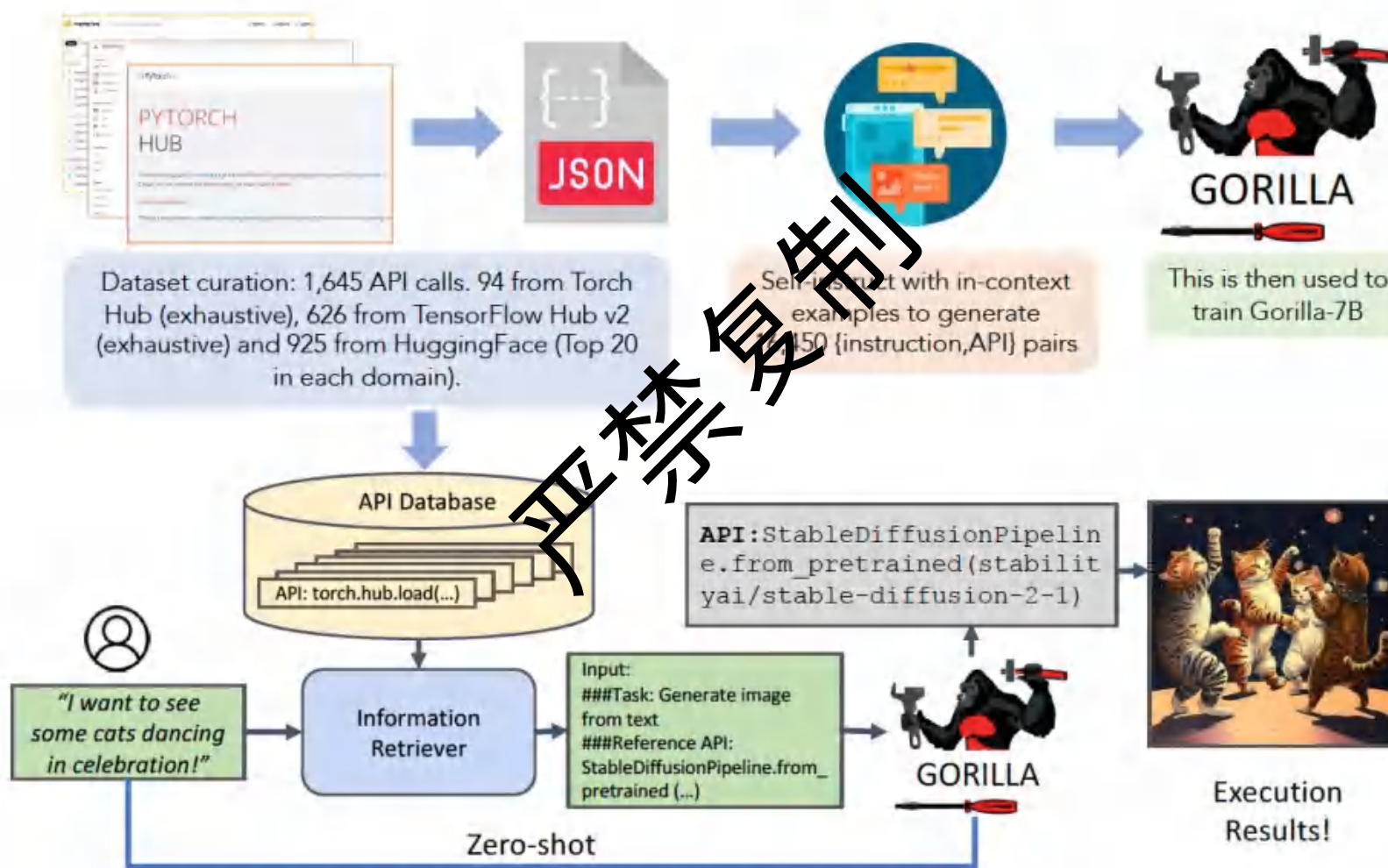
- 显著提升LLM在需要外部知识或计算任务上的表现。
- 减少LLM的幻觉，依赖工具获取准确信息。
- 具有良好的泛化能力，可学习使用新的工具。

Toolformer Agent – 使用场景

- 数学问题求解：
 - 调用计算器API。
- 实时信息查询：
 - 调用搜索引擎API获取最新新闻或数据。
- 日程管理：
 - 调用日历API查询或创建事件。
- 机器翻译：
 - 调用专业翻译API提升质量。
- 代码生成与执行：结合代码解释器API。
 - 相关工作: Gorilla (Patil et al., 2023) 专注于海量API调用。

禁止复制

Gorilla: 更善于写API Call的LLM



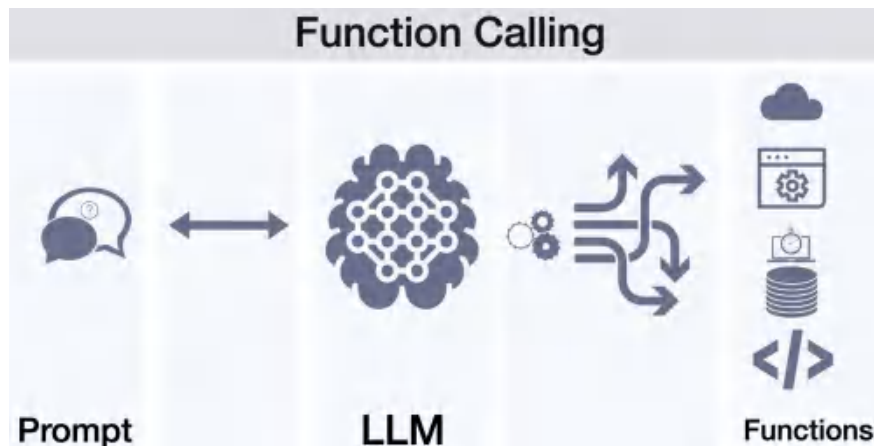
Toolformer Agent – 技术难点

- 工具选择：
 - 如何准确判断何时需要哪个工具。
- API参数生成：
 - 如何为选定的API生成正确的参数。
- API输出理解：
 - 如何解析和利用API返回的结果。
- 大量API的管理与学习：
 - 如何高效处理成百上千的API。
- 安全性：
 - 调用外部API可能带来的安全风险。

严禁复制

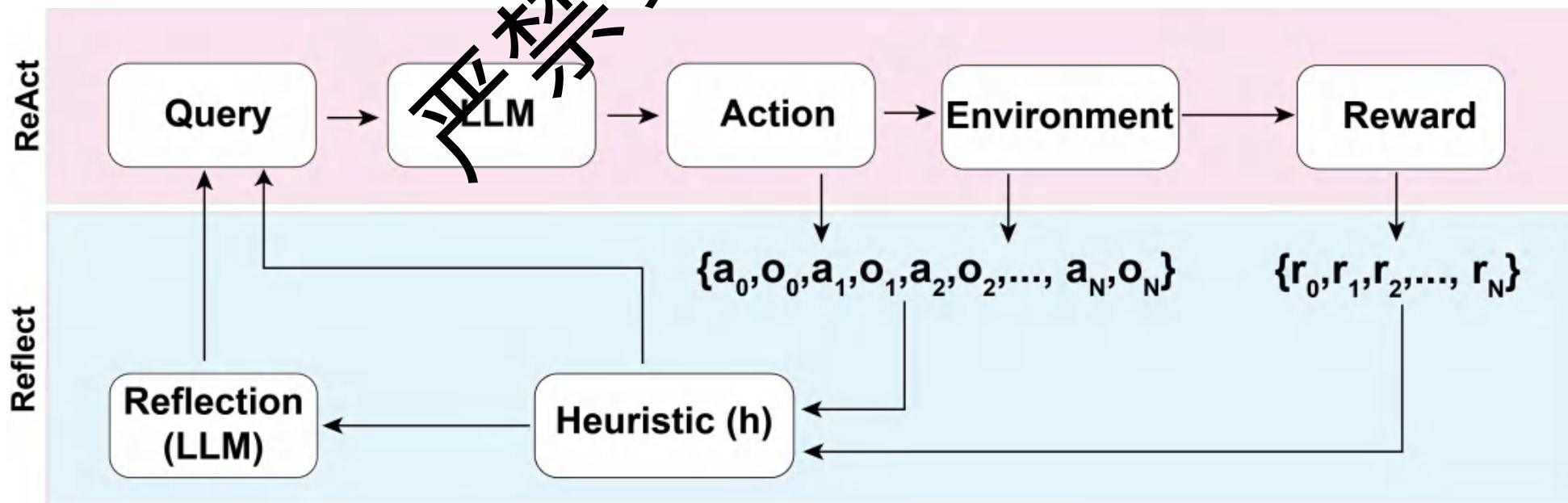
Toolformer和Function calling的区别

- 学习机制：
 - Toolformer 侧重于模型如何自主地、自监督地学习工具的使用，将工具调用结果内化。
 - Function Calling通过显式训练（通常是监督学习或强化学习）LLM学会输出一个结构化的工具请求。
- 集成方式：
 - Toolformer 是工具调用和结果是模型生成过程的内部部分。
 - 而当前函数调用更多是一个外部循环：
 - 模型输出请求 -> 外部执行 -> 结果返回模型 -> 模型继续生成。
- 训练数据生成：
 - Toolformer 自动化生成。
 - Function calling需要人工设计来创建。
- 总结：
 - Toolformer是未来方向，Function calling是实用步骤



类型三：反思型Agent – Reflexion

- 核心思想：
 - Agent在行动后进行自我反思，评估结果。
 - 从失败经验中学习，动态调整后续策略。
 - 维护一个短期记忆（当前轨迹）和长期记忆（经验库）。
- 论文： Shinn, N. et al. (2023). Reflexion: an autonomous agent with dynamic memory and self-reflection.
- Heuristic: 何时反思
- Reflexion LLM:



Reflection LLM模型

- Reflection LLM 的核心输出是“反思内容”：
 - 接收当前的“轨迹”（trace）：过去的一系列观察、思考和行动。
 - 分析这个轨迹：找出失败的原因、无效的策略。
 - 提炼反思内容。
- 反思内容如何“更新记忆”：
 - 反思内容：会被存储到智能体的“动态记忆”（dynamic memory）中。
 - 例如：“我上次尝试用A方法失败了，因为B原因，下次应该尝试C方法”
 - 动态记忆：是一个可供未来查询和利用的知识库。
- 记忆如何影响新的 prompt：
 - 需要进行下一步行动时，它会从动态记忆中检索相关的信息。
 - 会被整合到发送给主 LLM（负责规划和行动的那个LLM）的 prompt 中。
 - 因此，主 LLM 在生成新的思考（thought）和行动（action）时，避免重犯。

例子

Trial #1: Question: Grown-Ups starred the actor who was best known for which role on "'Allo 'Allo!'?"

Thought 1: I need to search Grown-Ups and "'Allo 'Allo!", find the actor who starred in Grown-Ups, then find the role he was best known for in "'Allo 'Allo!'".

Action 1: Search[Grown-Ups]

Thought 2: Grown-Ups starred Lesley Manville, Philip Davis, Brenda Blethyn, Janine Duvitski, Lindsay Duncan and Sam Kelly. I need to search "'Allo 'Allo!" and find which actor was best known for which role.

Action 2: Search["'Allo 'Allo!"]

Observation 2: Could not find ["'Allo 'Allo!"]. Similar: ["'Allo 'Allo!", "List of 'Allo 'Allo! characters", 'Gorden Kaye', "'Allo 'Allo! (series 2)", 'Allô, allô', 'Francesca Gonshaw', "'Allo 'Allo! (series 4)", 'Kim Hartman', "'Allo 'Allo! (series 6)", 'Jeremy Lloyd']

Reflection: I searched the wrong title for the show, "'Allo 'Allo!", which resulted in no results.

I should have searched the show's main character, Gorden Kaye, to find the role he was best known for in the show.

Trial #2: Question: Grown-Ups starred the actor who was best known for which role on "'Allo 'Allo!'?"

Thought 1: I need to search the actor who was best known for a role on "'Allo 'Allo!" and find out what role they were best known for.

Action 1: Search[Grown-Ups]

Observation 1: Grown-Ups is a 1980 British BBC television film devised and directed by Mike Leigh. It stars Lesley Manville, Philip Davis, Brenda Blethyn, Janine Duvitski, Lindsay Duncan and Sam Kelly. It was edited by Robin Sales and produced by Louis Marks for the BBC, and originally shown on BBC 2 on 28 November 1980.

Thought 2: The paragraph does not mention the actor who was best known for a role on "'Allo 'Allo!". I need to search the actor's name instead.

Action 2: Search[Sam Kelly]

Observation 2: Roger Michael Kelly (19 December 1943 – 14 June 2014), known by the stage name Sam Kelly, was an English actor who appeared in film, television, radio and theatre. He is best known for his roles as Captain Hans Geering in 'Allo 'Allo!, Warren in Porridge, Sam in On the Up, and Ted Liversidge in Barbara.

Thought 3: Sam Kelly is best known for his role as Captain Hans Geering in "'Allo 'Allo!", so the answer is Captain Hans Geering.

Action 3: Finish[Captain Hans Geering]

Observation 3: Answer is CORRECT

Reflexion Agent – 功能与优势

- 功能：
 - 通过“试错–反思–改进”循环提升任务成功率。
 - 存储和利用过去的成功与失败经验。
 - 动态调整内部认知模型和行为策略。
- 优势：
 - 显著提高在复杂、长时程任务中的表现。
 - 减少重复犯错，学习更高效的解决路径。
 - 更接近人类解决问题的方式，具有更好的鲁棒性。

Reflexion Agent – 使用场景

- 编程与代码调试：生成代码，测试，根据错误反馈修改。
- 复杂决策制定：如AlphaGo中的蒙特卡洛树搜索与强化学习。
- 交互式游戏：在游戏中学习规则，优化策略以获胜。
- 科学发现：提出假设，设计实验，根据结果修正假设 – 技术难点**
- 有效的自我评估：如何让LLM准确判断自身行为的优劣。
- 反思内容的生成：如何生成有建设性的、指导后续行动的反思。
- 记忆管理：如何高效存储和检索相关经验。
- 探索与利用的平衡：避免陷入局部最优，鼓励探索新策略。
- “反思过载”：过多的反思可能导致效率低下。

Reflexion Agent – 解决方案思路

- 自我评估的Prompting:
 - 设计特定的Prompt引导LLM进行批判性评估。
- 结构化记忆:
 - 使用向量数据库等存储经验，并进行相似性检索。
- 分层反思:
 - 从具体错误到抽象经验的总结。
- 启发式探索策略:
 - 结合随机性或置信度低的探索。
- 限制反思深度或频率:
 - 在效果和效率间取舍。

严禁复制

类型四：记忆增强型Agent

- 核心挑战：LLM的上下文窗口有限，无法处理长时程交互。
- 目标：赋予Agent持续学习和记忆能力。
- 关键组成：
 - 短期记忆 (Working Memory)：当前对话的上下文。
 - 长期记忆 (Long-term Memory)：存储过去的交互、知识、经验。
- 主流方案：基于检索增强生成 (RAG) 的思路。

记忆增强型Agent – 架构

- 整体架构：
 - 输入 -> LLM（短期记忆）
 - LLM -> 记忆模块（存储/检索）
 - 记忆模块 -> LLM（增强上下文）
 - LLM -> 输出
- 记忆模块：
 - 通常使用向量数据库（e.g., Pinecone, Chroma）。
 - 文本被编码为向量，通过相似度检索相关记忆。

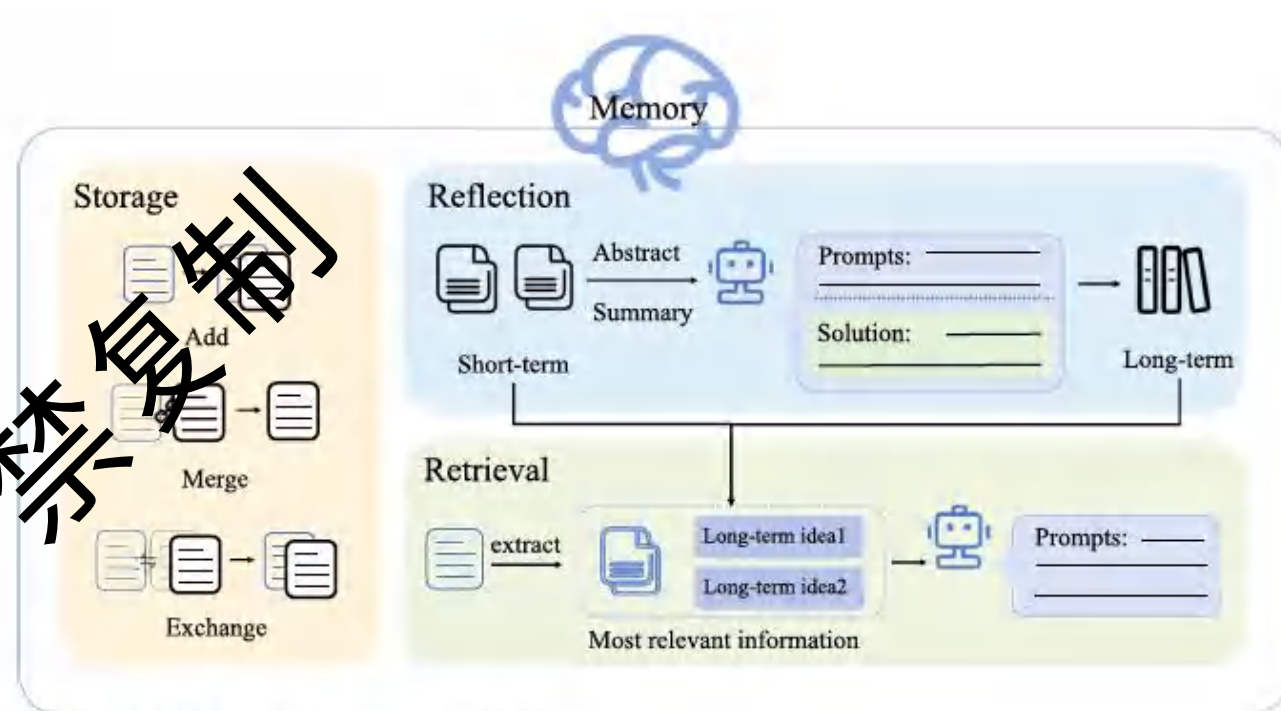


Fig. 2 The operational mechanism of the memory module

记忆增强型Agent – 功能与场景

- 功能：

- 保持对话连贯性，即使跨越多轮或长时间间隔。
- 实现个性化，记住用户的偏好和历史信息。
- 从过去的经验中学习，避免重复回答或犯错。

- 使用场景：

- 个性化虚拟助手：记住用户的习惯、日程、兴趣。
- 持续学习系统：如在线教育Agent，追踪学生学习进度。
- 需要大量背景知识的任务：如客服、领域专家系统。

如果没有记忆：

用户：Hi TravelMate, 我想计划一次去巴黎的旅行。

TravelMate：好的，您想什么时候去巴黎？

用户：下个月10号左右，待一周。

TravelMate：您从哪里出发？

用户：上海。

TravelMate：你要去哪里？

记忆增强型Agent – 技术难点

- 记忆的特征：
 - 如何有效地将信息转化为可存储和检索的形式。
- 检索的准确性与效率：
 - 如何在海量记忆中快速找到最相关信息。
- 记忆的更新与遗忘：
 - 如何处理过时信息，避免记忆冗余。
- 隐式记忆与显式记忆的结合：
 - 模型参数中的知识 vs. 外部记忆。
- 记忆的压缩与抽象：
 - 如何从原始信息中提炼关键知识。

严禁复制

记忆增强型Agent – 解决方案思路

- 表征：
 - Transformer编码器、句子嵌入模型。 (<https://arxiv.org/html/2405.06067v1>)
- 检索：
 - 近似最近邻搜索 (ANN)、混合检索 (关键词+向量)
- 更新与遗忘：
 - 基于时间衰减、相关性反馈、记忆评分机制。
- 压缩与抽象：
 - LLM本身进行总结、知识图谱构建。
- 分层记忆：
 - 原始记忆、摘要记忆、概念记忆。

类型五：多Agent系统 (MAS)

- 核心思想：
 - 由多个自主或半自主的Agent组成，通过协作解决复杂问题。
 - 每个Agent可以有不同的角色、能力和知识。
- LLM在MAS中的角色：
 - 作为单个Agent的“大脑”。
 - 作为协调者或通信中介。
- 代表性工作：
 - CAMEL (Li et al., 2023): 通过角色扮演促进Agent协作。
 - MetaGPT (Hong et al., 2023): 模拟软件公司流程的多Agent框架。

禁止复制

MAS概览

- 在一个多智能体系统中：
 - 初步步骤包括创建角色（profiles），为每个智能体赋予个性化的特征和子任务分配。
 - 智能体制定计划，感知环境，并从记忆中检索历史知识。
 - 利用大规模语言模型（LLM）的强大能力，智能体能够执行行动计划。
 - 智能体进行自我反思，优化决策。
 - 任务的执行依赖于智能体之间的互动，这些互动共同促进了整体任务的规划和实施。

Mutual Interaction



Group Planning

Question: 101 Park Avenue, are located in which city?

Thought: The question Simplified is find with city 101 Park are located in.

Action: Retrieval[101 Park Avenue]

Observation: 101 Park Avenue is in 36-story office building...

Thought: ...

Thought: I have something to say.

Action: Taking a bag

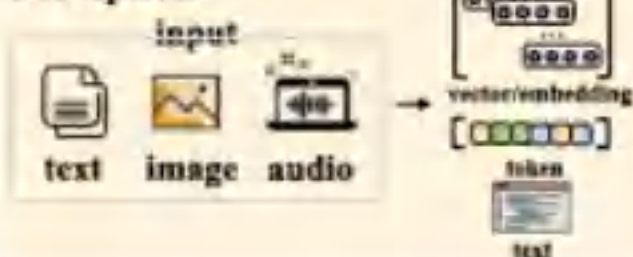
Observation: Please reflect your answer based on history.

Thought: My answer is correct

Action: Reflect[right]

Observation: The answer is CORRECT.

Perception



LLM-based Agent



Name: Jane

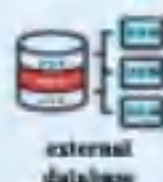
Profile

Description: She is a worker and she like cooking...

Intention: take chocolate

Self-Action

Knowledge



Memory



Reasoning & Planning

Trajectories

Action: We'd better try to catch...

first,
second,

Observation: Please reflect your answer

Thought: ...

Action:....

Reward: 1.0



Action: We'd better try to catch...

first,
second,

Observation: Please reflect your answer

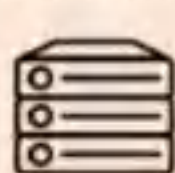
Thought: ...

Action:....

Reward: 0.0



Evolution



Memory



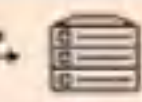
Filtered trajectories



reflection



storage



suitable



Plan-Agent



Tool-Agent



Reflex-Agent

message

Other Agent

多Agent系统 – 架构与范式

- 中心化 vs. 去中心化控制
- 协作模式：
 - 任务分解与分配：一个主Agent分配子任务。
 - 协商与投票：Agent间通过沟通达成共识。
 - 竞争与合作：类似博弈论中的场景。
 - 角色扮演：不同Agent扮演不同专家角色。

禁止复制

CAMEL框架详解

- 核心机制：

- 设定两个角色扮演的Agent（如AI用户和AI助手）。
- 通过“指令设计者Agent”生成初始任务和对话流程。
- 两个Agent轮流对话，共同完成任务。

- 例子

- 人类：开发一个股票市场的交易机器人
- 充当Python程序员的AI助手代理
- 充当股票交易员的AI用户代理
- 任务指定代理，形成一个明确的任务，供助手来解决
- AI用户和AI助手都会获得这个具体任务，相互交流，进行合作

禁止复制

Human Input

Idea: Develop a trading bot for the stock market



Human User

Role Assignment:

AI Assistant:
Python Programmer

AI User:
Stock Trader



Specified Task: Develop a trading bot with a sentiment analysis tool that can monitor social media platforms for positive or negative comments about a particular stock, and execute trades based on sentiment analysis results



AI Assistant

Role Playing Session

Instruction: Install the necessary Python libraries for sentiment analysis and stock trading.
Input: None

Solution: To install the necessary Python libraries for sentiment analysis and stock trading, we can use pip, the package installer for Python. Here are the libraries we need to install:

1. tweepy - for accessing the Twitter API
2. textblob - for sentiment analysis
3. yfinance - for accessing Yahoo Finance API

To install these libraries, open the command prompt or terminal and enter the following commands:

```
pip install tweepy
pip install textblob
pip install yfinance
```

Next request.

Instruction: Import the necessary libraries in Python.
Input: None

Solution: Here's the code to import these libraries:

```
...
import tweepy
from textblob import TextBlob
import pandas as pd
import numpy as np
import yfinance as yf
...
```

Next request.

Assistant System Prompt:

Never forget you are a <ASSISTANT_ROLE> and I am a <USER_ROLE>.
Never flip roles! Never instruct me!
We share a common interest in collaborating to successfully complete a task.
You must help me to complete the task.
Here is the task: <TASK>. Never forget our task!
I must instruct you based on your expertise and my needs to complete the task.

I must give you one instruction at a time.
You must write a specific solution that appropriately completes the requested instruction.

You must decline my instruction honestly if you cannot perform the instruction due to physical, moral, legal reasons or your capability and explain the reasons.
Unless I say the task is completed, you should always start with:

Solution: <YOUR_SOLUTION>

<YOUR_SOLUTION> should be specific, and provide preferable implementations and examples for task-solving.

Always end <YOUR_SOLUTION> with: Next request.

Task Specifier Prompt:

Here is a task that <ASSISTANT_ROLE> will help <USER_ROLE> to complete: <TASK>.

Please make it more specific.

Be creative and imaginative.

Please reply with the specified task in <WORD_LIMIT> words or less.

Do not add anything else.

User System Prompt (经理)

Never forget you are a <USER_ROLE> and I am a <ASSISTANT_ROLE>.

Never flip roles! You will always instruct me.

We share a common interest in collaborating to successfully complete a task.

I must help you to complete the task.

Here is the task: <TASK>. Never forget our task!

You must instruct me based on my expertise and your needs to complete the task ONLY in the following two ways:

1. Instruct with a necessary input:

Instruction: <YOUR_INSTRUCTION>

Input: <YOUR_INPUT>

2025-6-9

The "Instruction" describes a task or question. The paired "Input" provides further context or information for the requested "Instruction".

You must give me one instruction at a time.

I must write a response that appropriately completes the requested instruction.

I must decline your instruction honestly if I cannot perform the instruction due to physical, moral, legal reasons or my capability and explain the reasons.

You should instruct me not ask me questions.

Keep giving me instructions and necessary inputs until you think the task is completed.

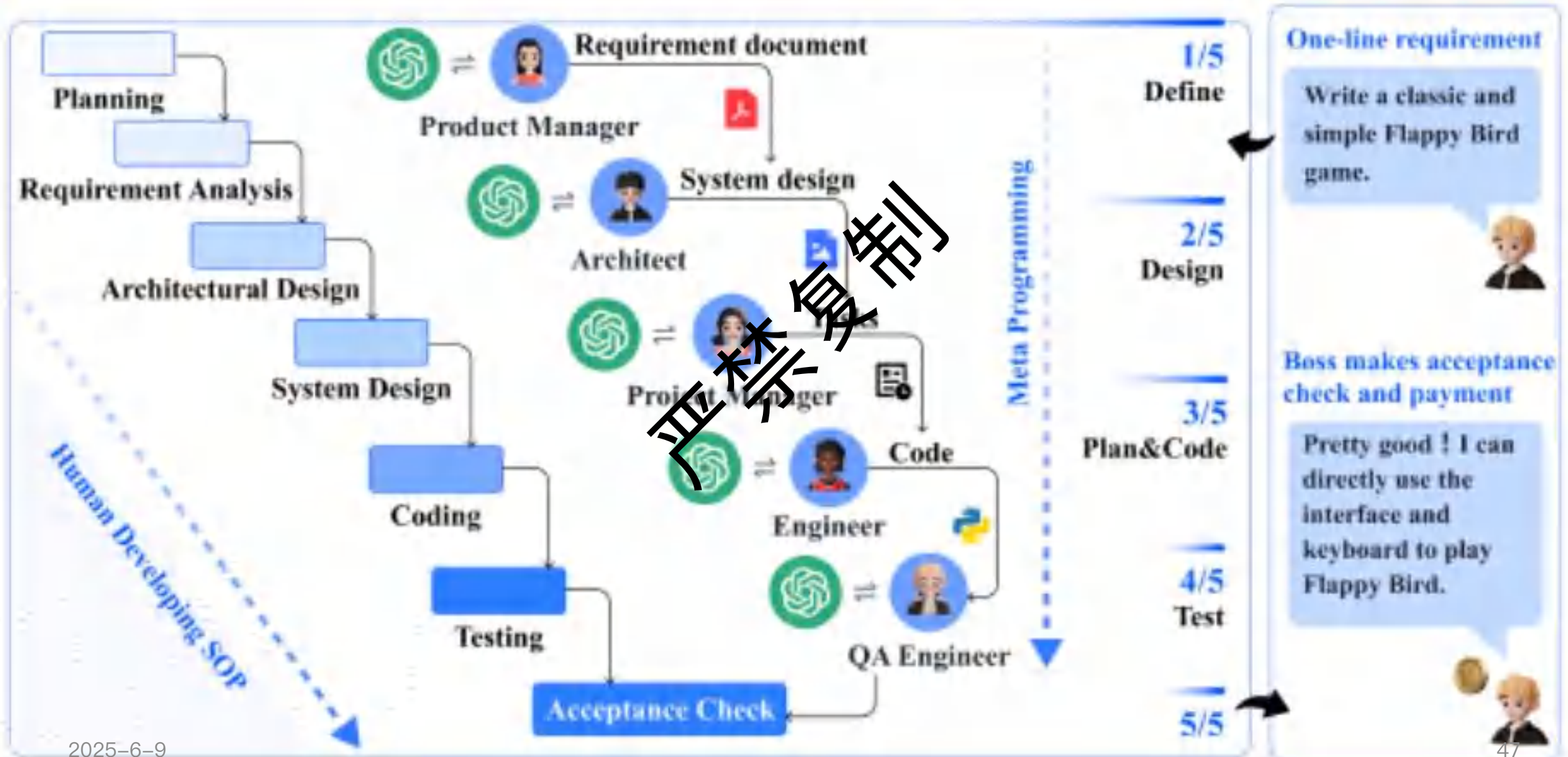
When the task is completed, you must only reply with a single word <CAMEL_TASK_DONE>.

MetaGPT框架详解

- 核心思想：
 - 通过标准化流程，将复杂任务分解成多个简单任务，通过多Agent协作实现。
- Agent角色：
 - 产品经理、架构师、工程师、测试工程师等。
- 工作流程：
 - 需求分析 -> 架构设计 -> 代码生成 -> 测试与调试。
 - Agent间通过标准化的文档（SOPs）和消息传递进行协作。

MetaGPT Agents Collaboration with Developing SOP

Human interaction





Make the 2048 sliding tile number puzzle game

Human Input

MetaGPT



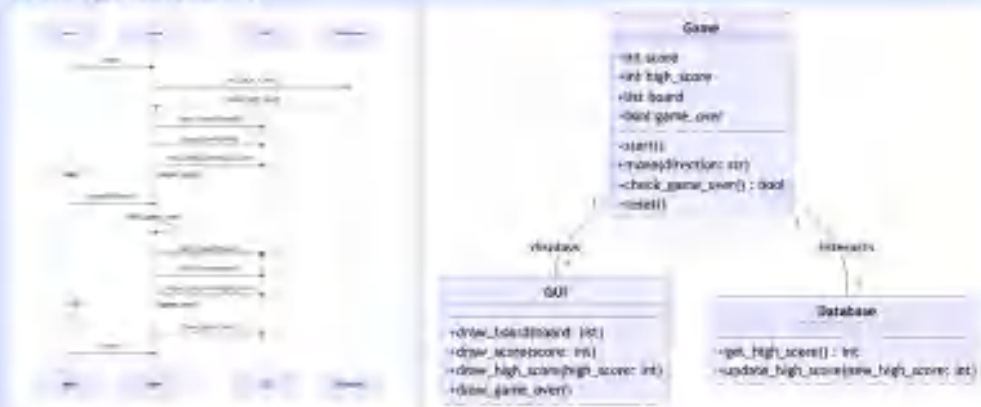
Architect

CTO

Action: Write Design

Think: For this project, we will use Pygame, an open-source library designed for making video games with Python.

□ Program call flow



File list

["main.py", "game.py", "gui.py", "database.py",
"test_game.py", "test_gui.py", "test_database.py"]



Product Manager

PM

Action: Write PRD

Think: The boss requires the creation of 2048 sliding tile number puzzle game.

Product Goals

"Create a web-based 2048 game that is easy to play"

Incorporate features that enhance user engagement and retention

User Stories

"As a user, I want to easily understand how to play the game so that I can start playing immediately".

"As a user, I want the game to be engaging so that I can play it for a long time"...

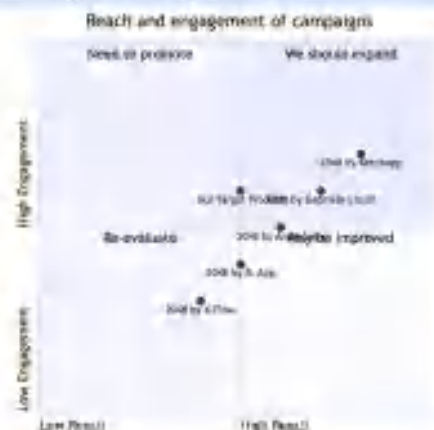
Competitive Analysis

2048 by Gabriele Cirulli: The original 2048 game, simple and easy...

2048 by Ketchapp: Offers a variety of game modes...

2048 by Androbaby: Has a clean interface and high score tracking...

Competitive Quadrant Chart



Requirement Analysis

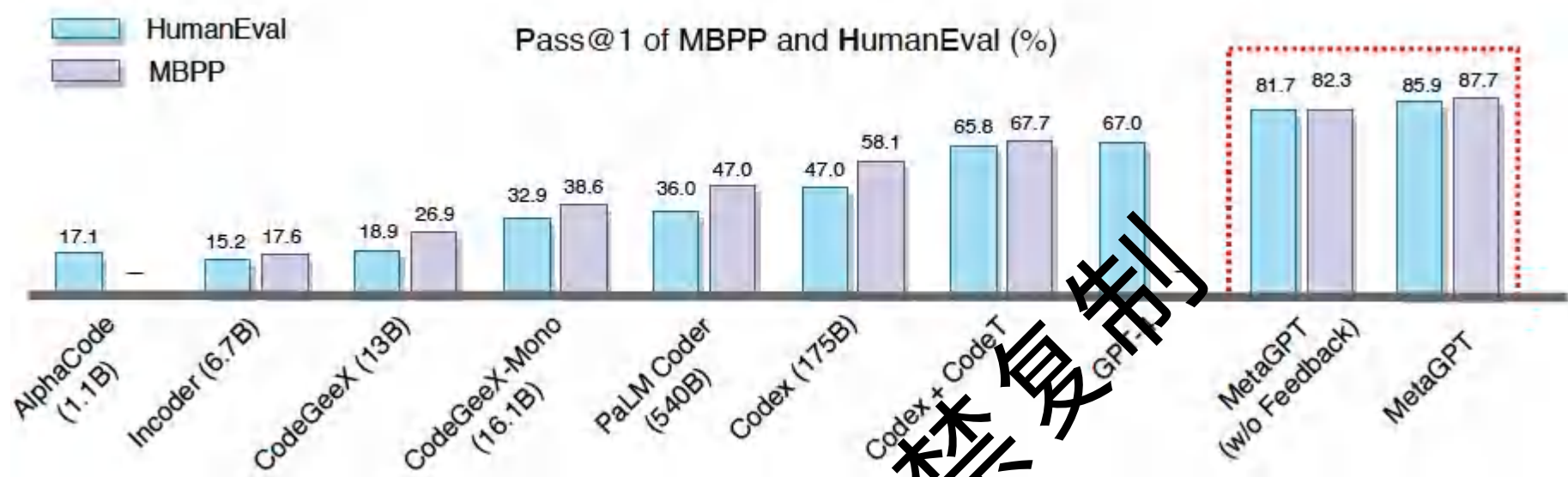
The product should be a 2048 sliding tile number puzzle game which is user-friendly.

Requirement Pool

"Develop a simple and intuitive user interface", "P0".

"Implement engaging gameplay mechanics", "P0".

实施效果和消融实验



Engineer	Product	Architect	Project	#Agents	#Lines	Expense	Revisions	Executability
✓	✗	✗	✗	1	83.0	\$ 0.915	10	1.0
✓	✓	✗	✗	2	112.0	\$ 1.059	6.5	2.0
✓	✓	✓	✗	3	143.0	\$ 1.204	4.0	2.5
✓	✓	✗	✓	3	205.0	\$ 1.251	3.5	2.0
✓	✓	✓	✓	4	191.0	\$ 1.385	2.5	4.0

多Agent系统 – 功能与优势

- 功能：

- 处理单Agent难以解决的超复杂问题。
- 模拟真实世界中的组织和社会行为。
- 通过分工与协作提高效率和鲁棒性。

- 优势：

- 专业化：每个Agent可以专注于特定子任务或领域。
- 并行性：多个Agent可以同时工作。
- 可扩展性：易于增加或替换Agent。
- 涌现智能：可能产生单个Agent不具备的集体智能。

禁止复制

多Agent系统 – 使用场景

- 复杂软件工程自动化 (如MetaGPT)。
- 大规模社会模拟：经济模型、城市交通、舆情分析。
- 科学研究：分布式实验设计、数据分析、论文撰写。
- 交互式娱乐：动态生成复杂剧情、多角色NPC。
- 分布式决策支持系统。

多Agent系统 – 技术难点

- 通信协议设计：Agent间如何有效交换信息。
- 协调与同步：如何确保Agent行动的一致性和有序性。
- 冲突解决：当Agent意见不一时如何处理。
- 可信度与责任界定：当系统出错时，如何追溯。
- “社会性幻觉”：Agent间可能产生无效或冗余的交互。

多Agent系统 – 解决方案思路

- 标准化的通信语言和格式（如基于JSON或XML）。
- 引入协调者Agent或全局规划器。
- 设计明确的角色和责任。
- 基于声誉或投票的决策机制。
- 分层架构，高层Agent管理低层Agent。
- 严格的日志和监控机制。



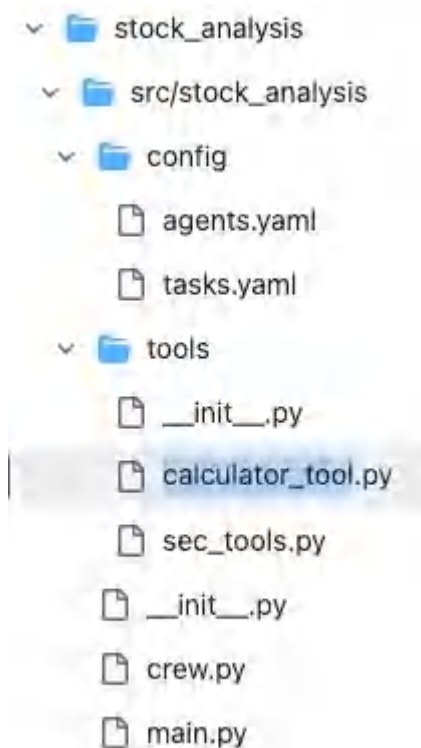
04 复制 Agent 实战

LLM 代理的关键组件

- 虽然架构各有不同，但常见的组件包括：
 - LLM 核心（“大脑”）：负责理解、推理、规划以及生成响应或行动的核心 LLM。
 - 记忆模块：
 - 短期记忆：LLM 的上下文窗口，用于当前任务的临时存储区域。
 - 长期记忆：向量数据库、传统数据库或文件系统，用于持久化知识和过往经验。
 - 规划模块：将复杂目标分解为更小、更易管理的步骤或任务。可能涉及生成子目标、思维链或自我批评。
 - 行动模块 / 工具：使代理能够与环境交互。
 - 示例：代码执行（如 Python 解释器）、网络搜索、API 调用、数据库查询、文件系统操作。
 - 感知模块（可选但常见）：代理如何从环境中接收超出用户直接输入的信息（例如，观察工具输出、读取文件）。

实现stock分析的agent

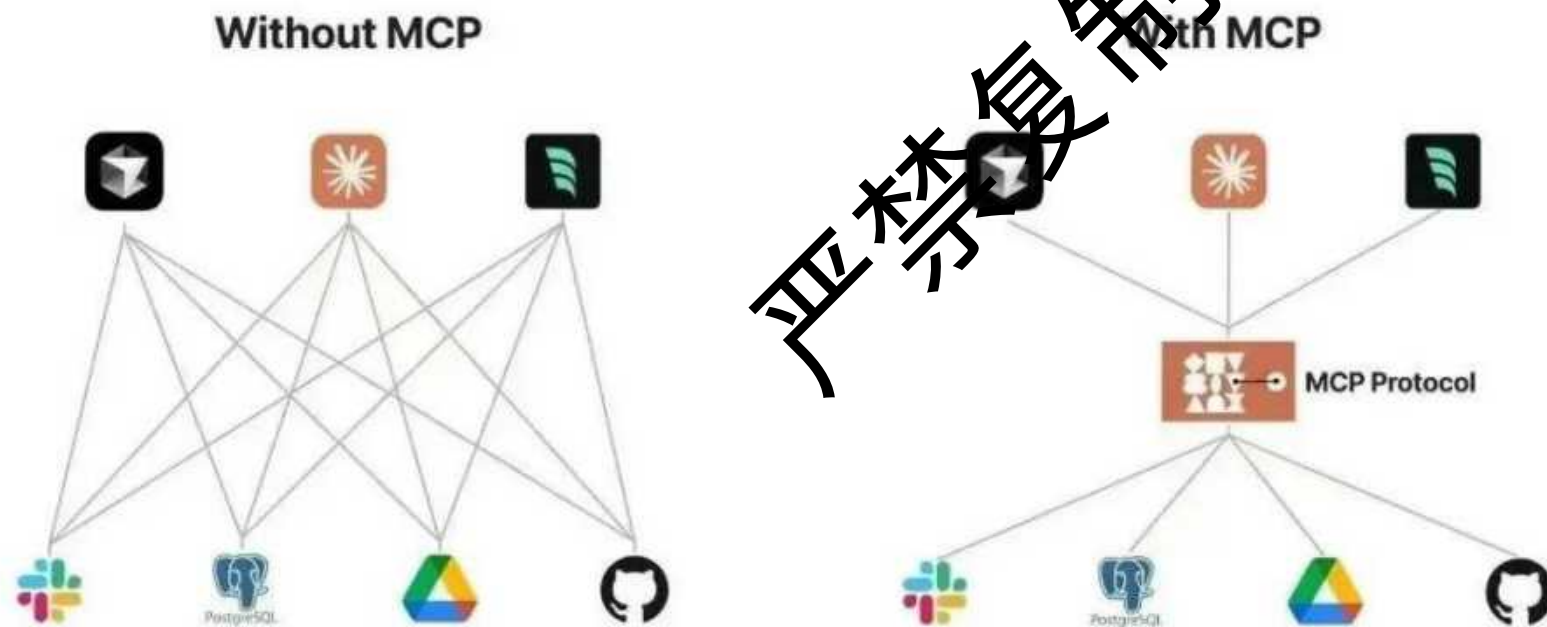
- 代码路径：
 - https://github.com/crewAIInc/crewAI-examples/tree/main/stock_analysis
- 模块：



严禁复制

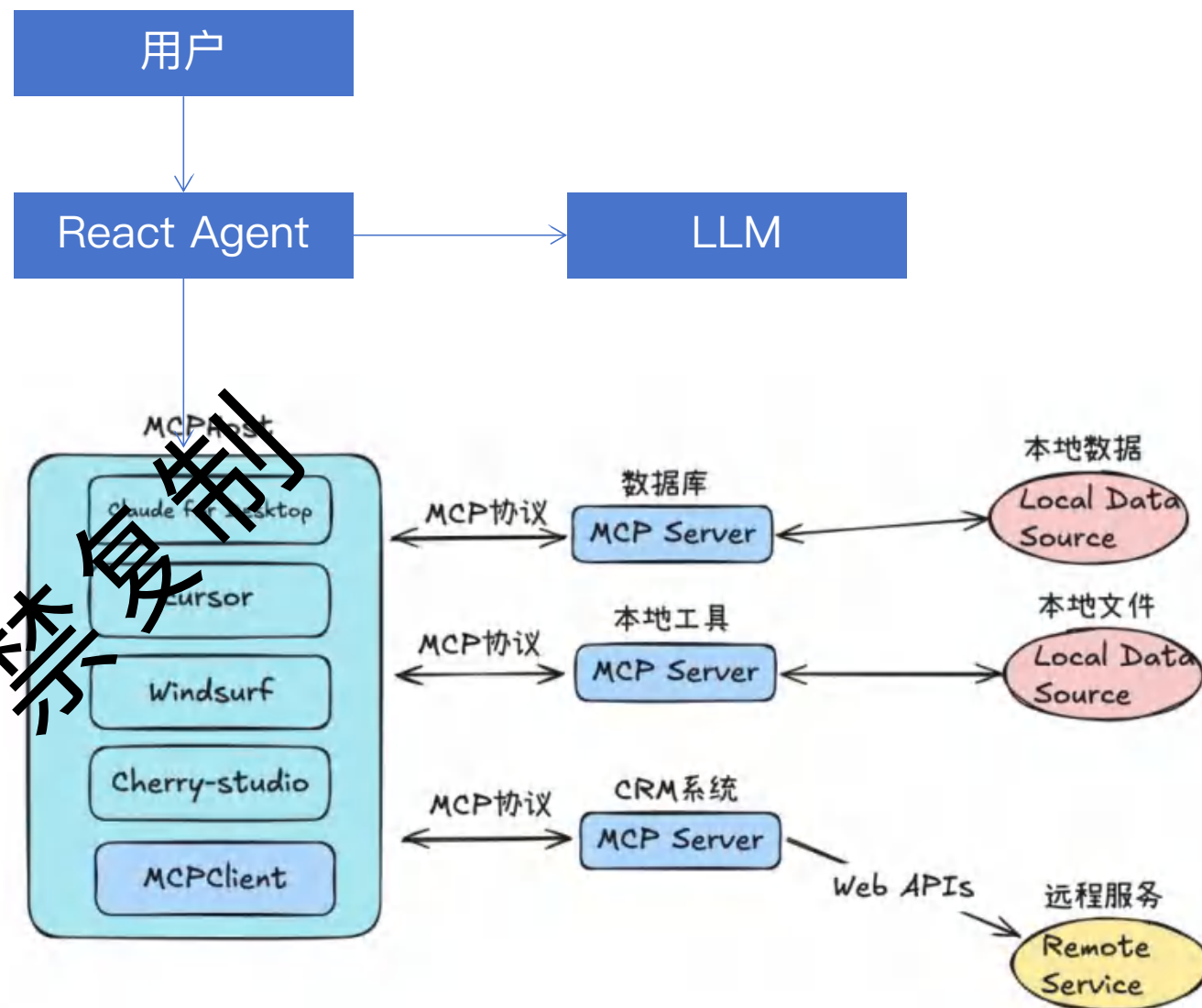
MCP (Model Context Protocol) 模型上下文协议

- <https://mp.weixin.qq.com/s/kgqi8oYRS2sw2wZT4EHjsQ>
- 解决的问题:



LLM和MCP的协作

1. 用户把问题提给react agent;
2. 用户的查询 + 工具描述 一起发送给 LLM (如Claude) ;
3. LLM (如Claude) 决定使用哪些工具 (如果有的话) ;
4. agent通过server执行工具调用;
5. agent对返回结果进行react (多次调用 LLM) ;
6. agent把结果显示给用户。



<https://medium.com/@romanbessouat/how-i-built-a-coding-assistant-using-mcp-server-and-llm-agent-099a338fdf9e>

react agent使用的prompt

You are a coding assistant expert. When responding to a question, please follow these steps:

How To Answer User Query

Step 1: Review the Project Structure

Step 2: Inspect and Gather Information

Step 3: Outline Your Approach

Before providing an answer, describe the steps and thought process you will take to address the question.

Explain your reasoning regarding what parts of the project you targeted, what issues or opportunities for improvement you identified, and what changes you propose.

Step 4: Provide a Complete Code Implementation

Step 5: Summarize Your Recommendations

IMPORTANT

Use any tools available to explore the project structure and inspect files to ensure your analysis is thorough before producing your final code and explanation. """

引用的主要综述性论文

1. Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., ... & Wang, L. (2023). The Rise and Potential of Large Language Model Based Agents: A Survey. arXiv preprint arXiv:2309.07864.
2. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... & Zhou, J. (2023). A survey on Large Language Model based Autonomous Agents. arXiv preprint arXiv:2308.11432.
3. Qiao, S., Wang, Y., Wang, C., Dou, Y., & Wang, B. (2023). A Survey of Autonomously Improving Large Language Models. arXiv preprint arXiv:2311.11899.
4. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629.
5. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., ... & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.
6. Shinn, N., Labash, B., & Gopinath, A. (2023). Reflexion: an autonomous agent with dynamic memory and self-reflection. arXiv:2303.11366.
7. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., ... & Nyberg, E. (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651.
8. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.
9. Li, G., Mendi, A. F., Akkiraju, R., Lin, C., Kamar, E., Sahni, V., & Yu, T. (2023). CAMEL: Communicative Agents for “Mind” Exploration of Large Scale Language Model Society. arXiv:2303.17760.
10. Hong, S., Zheng, X., Chen, J., Cheng, Y., Zhang, C., Wang, Z., ... & Chan, A. (2023). MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. arXiv:2308.00352.
11. Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. (此为基于小模型的生成式Agent，但影响深远)

THANK YOU

北京交通大学软件学院

严禁复制

